

COVER PAGE

CST2550 Reset Coursework Report

Local Supermarket Management System for Small Shops

Student Name: Kallakuri Sri Varshitha Vyshnavi

MDX ID: M01053500

Date: 04 July 2026

Module Leader: Ahmed Eissa

University: Middlesex University London

TABLE OF CONTENTS

- Introduction
- Justification of Data Structures and Algorithms
- Time Complexity Analysis with Pseudocode
- Testing Approach and Test Cases
- Conclusion, Limitations, and Reflection
- References

Introduction

Small supermarkets often rely on manual paper-based systems to manage products, suppliers, stock levels, and sales. These methods are slow, prone to errors, and make searching or reporting difficult. As small shops grow, they require a digital solution that improves accuracy, reduces manual effort, and supports efficient daily operations.

The Local Supermarket Management System developed for this coursework aims to replace manual workflows with a structured digital system built using C# .NET, SQL Server, and Entity Framework. The system supports product management, supplier records, stock control, sales processing, searching, and reporting. It allows shop staff to add, update, remove, and search products using details such as product ID, name, category, supplier, barcode, price, and quantity in stock.

This report explains the design choices, selected data structures, algorithms, time complexity analysis, testing approach, and reflections on how effectively the system supports small supermarket operations.

Justification of Selected Data Structures and Algorithms

The system requires custom data structures to manage products, suppliers, stock quantities, and sales records. These structures were selected to balance efficiency, simplicity, and suitability for small-shop environments.

A custom array-based structure was used to store products. It maintains an internal array and a counter to track the number of stored items. This structure supports adding, removing, updating, and searching products. It was chosen because it is simple to implement, provides predictable memory layout, and is suitable for small datasets typically found in small supermarkets.

Binary search was selected for barcode lookup. Barcodes are unique identifiers, and searching for them is a frequent operation in supermarket systems. Binary search provides fast performance and works effectively when products are stored in a sorted array.

Linear search was used for product name searches. Product names are not unique, and staff may search using partial names. Linear search supports flexible matching and is appropriate for small datasets.

Sales and stock structures were designed using separate objects for sales and sale items. This mirrors real supermarket receipts and allows accurate tracking of quantities sold and remaining stock. Stock updates use direct access to product objects, ensuring efficient quantity changes.

Time Complexity Analysis with Pseudocode

Add Product

Code

```
function AddProduct(product):  
    if count == array.length:  
        resize array  
    array[count] = product  
    count = count + 1
```

This operation runs in constant time on average, with occasional linear time when resizing occurs.

Search by Name (Linear Search)

Code

```
function SearchByName(name):  
    for i from 0 to count-1:  
        if array[i].Name == name:  
            return array[i]  
    return null
```

This operation has linear time complexity because each product may need to be checked.

Search by Barcode (Binary Search)

Code

```
function SearchByBarcode(barcode):  
    left = 0
```

```

right = count - 1
while left <= right:
    mid = (left + right) / 2
    if array[mid].Barcode == barcode:
        return array[mid]
    else if barcode < array[mid].Barcode:
        right = mid - 1
    else:
        left = mid + 1
return null

```

Binary search has logarithmic time complexity, making it efficient for frequent barcode lookups.

Update Stock

Code

```

function UpdateStock(id, qty):
    product = FindById(id)
    if product != null:
        product.Quantity = qty

```

The complexity depends on the search method used. Linear search results in $O(n)$, while binary search results in $O(\log n)$.

Low-Stock Report

Code

```

function LowStockReport(threshold):
    result = empty list
    for i from 0 to count-1:
        if array[i].Quantity < threshold:
            add array[i] to result
    return result

```

This operation requires checking all products and therefore has linear time complexity.

Testing Approach and Test Cases

The testing approach included unit testing, validation testing, database testing, and functional testing. Unit tests were used to verify core supermarket operations such as adding products, updating stock, searching, and recording sales. Validation tests ensured that data such as barcodes,

prices, and quantities met required constraints. Database tests confirmed that Entity Framework correctly saved and retrieved data. Boundary tests were used for low-stock alerts and empty datasets. Functional tests ensured that menu navigation and user input were handled correctly.

Example Test Cases

- Adding a product with valid details should store the product successfully.
- Adding a product with a duplicate barcode should produce an error.
- Searching by name should return matching products.
- Searching by barcode should return the correct product.
- Updating stock should change the quantity accurately.
- Low-stock alerts should list products below the threshold.
- Recording a sale should create a sale record and reduce stock.
- Negative prices or quantities should trigger validation errors.
- Adding a supplier should store supplier details correctly.
- Generating category reports should list products belonging to that category.

Conclusion, Limitations, and Reflection

The Local Supermarket Management System successfully replaces manual processes with a digital solution that improves accuracy, searching, stock visibility, and reporting. Custom data structures ensure efficient searching, while SQL Server and Entity Framework provide reliable data storage and persistence.

The system has some limitations. The console interface is basic and lacks visual appeal. The system does not include barcode scanner integration, receipt printing, customer accounts, or advanced analytics. Error handling is limited for complex scenarios.

Developing this system improved understanding of C# .NET, data structures, algorithms, SQL Server, and Entity Framework. Implementing custom structures strengthened algorithmic thinking, while building reports and sales workflows provided insight into real retail systems. Future improvements could include a graphical interface, barcode scanning, cloud deployment, and enhanced reporting features.

REFERENCES

Microsoft (2024) *Entity Framework Core Documentation*. Microsoft (2024) *C# Programming Guide*. Robinson, J. (2023) *Data Structures and Algorithms in C#*. Elmasri, R. and Navathe, S. (2022) *Fundamentals of Database Systems*.